ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC – KỸ THUẬT MÁY TÍNH



Vi xử lý - Vi điều khiển

Báo cáo bài thực hành 2

Ngắt Timer và quét LED

GVHD : Phan Văn Sỹ

SV thực hiện : Trần Minh Quang – 2412855

TP. HỒ CHÍ MINH, THÁNG 10/ 2026

Mục lục

1 Cấu hình ngắt Timer **1**

2 Bài tập 1 **1**

3 Bài tập 2 **3**

4 Bài tập 3 **6**

5 Bài tập 4 **8**

6 Bài tập 5 **9**

7 Bài tập 6 **11**

8 Bài tập 7 **13**

9 Bài tập 8 **15**

10 Bài tập 9 **18**

11 Bài tập 10 **21**

12 Mã nguồn cho bài thực hành 2 **23**

1 Cấu hình ngắt Timer

Các bài tập trong bài thực hành này đều sử dụng ngắt của Timer 2 (TIM2) trên vi điều khiển STM32F103C6. Hệ thống dùng nguồn xung nội 8MHz. Với bài tập 1 đến 8, bộ chia trước (Prescaler) được đặt là 7999 và giá trị nạp lại (Counter Period) là 9, nên chu kỳ ngắt được tính như sau:

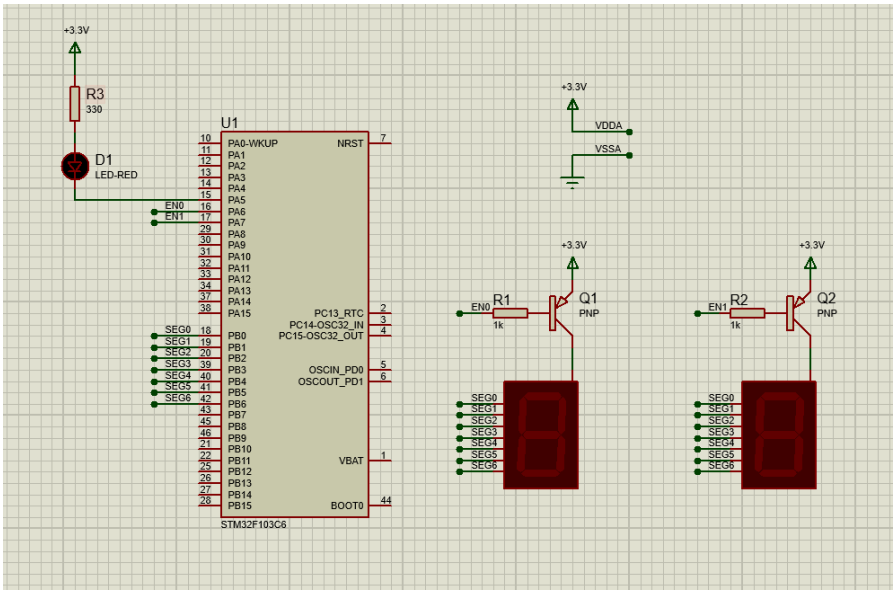
$$f_{TIM} = \frac{8\,\text{MHz}}{7999+1} = 1000\,\text{Hz}, \qquad f_{\text{ngắt}} = \frac{1000\,\text{Hz}}{9+1} = 100\,\text{Hz} \;\Rightarrow\; T_{\text{ngắt}} = 10\,\text{ms}$$

Timer được khởi động ở chế độ ngắt bằng lệnh `HAL_TIM_Base_Start_IT(&htim2)`, đặt trong vùng `USER CODE BEGIN 2`. Mỗi 10ms, hàm `HAL_TIM_PeriodElapsedCallback` được gọi một lần. Riêng bài tập 9–10 sử dụng chu kỳ ngắt 5ms để tăng tần số quét ma trận LED (xem mục Bài tập 9).

2 Bài tập 1

Bài tập này thực hiện giao tiếp giữa nhiều LED 7 đoạn và vi điều khiển STM32F103C6. Các LED 7 đoạn là loại Anode chung (7SEG-COM-ANODE). Các chân tín hiệu (cathode) của hai LED được nối chung vào PB0–PB6. Chân Anode của mỗi LED được cấp nguồn qua một transistor PNP, điều khiển bởi các chân EN0 (PA6) và EN1 (PA7). Tại mỗi thời điểm chỉ có một LED được bật. Yêu cầu là hiển thị số **"1"** trên LED thứ nhất và số **"2"** trên LED thứ hai, thời gian chuyển giữa hai LED là nửa giây.

Report 1: Sơ đồ mô phỏng trên Proteus:



Hình 1: Sơ đồ nguyên lý cho bài tập 1

Link github cho file mô phỏng bài tập 1: https://github.com/qanetta/Microprocessor-Lab/blob/main/lab2/proteus_files/Ex1_2-7seg.pdsprj

Report 2: Hàm ngắt `HAL_TIM_PeriodElapsedCallback` được gọi mỗi 10ms. Biến `counter` đếm số lần ngắt; sau 50 lần (500ms), hệ thống chuyển sang LED 7 đoạn còn lại. Mỗi lần chuyển, transistor của LED đang sáng được tắt trước, số cần hiển thị được xuất ra PB0–PB6, sau đó transistor của LED kế tiếp mới được mở, nhằm tránh hiện tượng bóng mờ (ghosting).

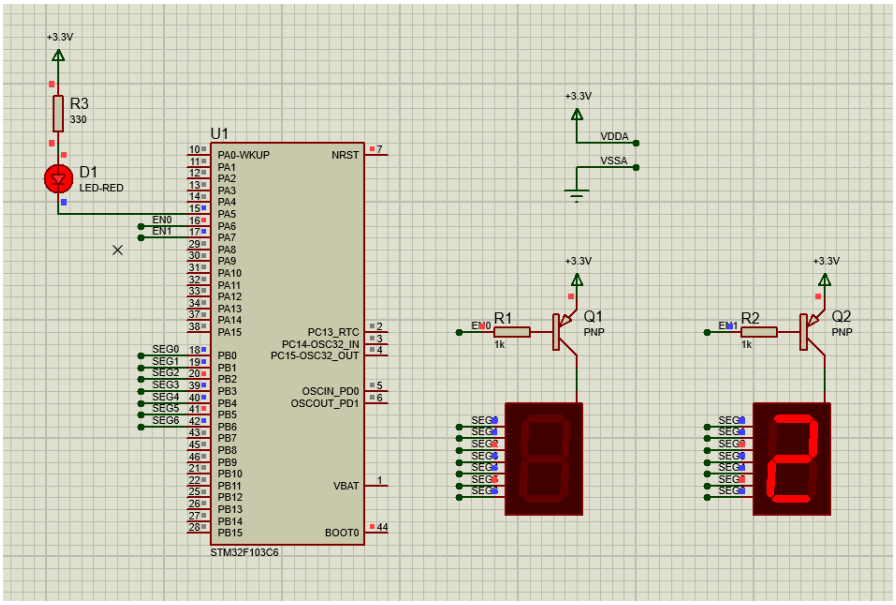
```
/* USER CODE BEGIN 4 */
int counter = 50;   // 50 x 10ms = 500ms
int status = 0;     // 0: bat LED 1, 1: bat LED 2

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){
    if (htim->Instance == TIM2) {
        counter--;
        if (counter <= 0) {
            counter = 50; // nap lai 500ms
            HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
            switch (status) {
                case 0:
                    // Tat LED 2
                    HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, GPIO_PIN_SET);
                    display7SEG(1);
                    // Mo transistor Q1 cho LED 1
                    HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, GPIO_PIN_RESET);
                    status = 1;
                    break;
                case 1:
                    // Tat LED 1
                    HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, GPIO_PIN_SET);
                    display7SEG(2);
                    // Mo transistor Q2 cho LED 2
                    HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, GPIO_PIN_RESET);
                    status = 0;
                    break;
                default:
                    status = 0;
                    break;
            }
        }
    }
}
/* USER CODE END 4 */
```

Mã nguồn ngắt timer cho bài tập 1

Hàm **display7SEG(int num)** được sử dụng lại từ bài thực hành 1. Trong hàm main, sau khi khởi động timer bằng `HAL_TIM_Base_Start_IT(&htim2)` và tắt cả hai transistor PNP, vòng lặp `while(1)` được để trống vì toàn bộ xử lý đã nằm trong ngắt timer.

Kết quả mô phỏng:



Hình 2: Kết quả mô phỏng bài tập 1

Câu hỏi ngắn: *Tần số quét LED là bao nhiêu?*

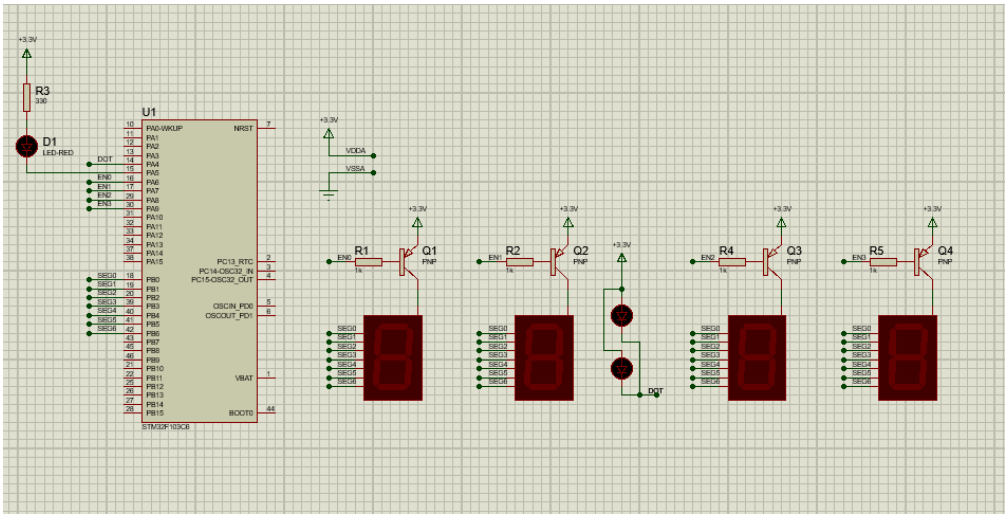
Mỗi LED sáng trong $T_S = 0.5$s và có 2 LED, nên chu kỳ quét toàn bộ là $2 \times 0.5 = 1$s. Tần số quét là:

$$f = \frac{1}{2 \cdot T_S} = \frac{1}{2 \times 0.5} = 1\,\text{Hz}$$

3 Bài tập 2

Mở rộng hệ thống lên 4 LED 7 đoạn (EN0–EN3) và thêm hai LED đơn ở giữa, gọi chung là **DOT**, nối với chân PA4. Hai LED DOT nhấp nháy mỗi giây. Bốn LED 7 đoạn lần lượt hiển thị **1**, **2**, **3**, **0** (biểu diễn 12 giờ 30 phút). Thời gian chuyển giữa các LED 7 đoạn là 500ms. Toàn bộ xử lý được hiện thực trong hàm ngắt timer.

Report 1: Sơ đồ mô phỏng trên Proteus:



Hình 3: Sơ đồ nguyên lý cho bài tập 2

Link github cho file mô phỏng bài tập 2 (dùng chung cho bài tập 2 đến 8): https://github.com/qanetta/Microprocessor-Lab/blob/main/lab2/proteus_files/Ex2-8_4-7seg-dot.pdsprj

Report 2: Hàm ngắt `HAL_TIM_PeriodElapsedCallback` được gọi mỗi 10ms và dùng hai bộ đếm độc lập: `dot_counter` (100 lần ngắt = 1s) để nhấp nháy DOT, và `seg_counter` (50 lần ngắt = 500ms) để chuyển giữa 4 LED 7 đoạn. Biến `led7seg_state` lưu chỉ số của LED sẽ được bật ở lần chuyển kế tiếp. Mỗi lần chuyển, 3 LED còn lại được tắt trước, số tương ứng được xuất ra PB0–PB6, sau đó transistor của LED cần bật mới được mở.

```
/* USER CODE BEGIN 4 */
int seg_counter = 50;      // 50 x 10ms = 500ms: chuyen LED 7 doan
int dot_counter = 100;     // 100 x 10ms = 1s: nhay DOT
int led7seg_state = 0;     // LED 7 doan dang sang (0..3)

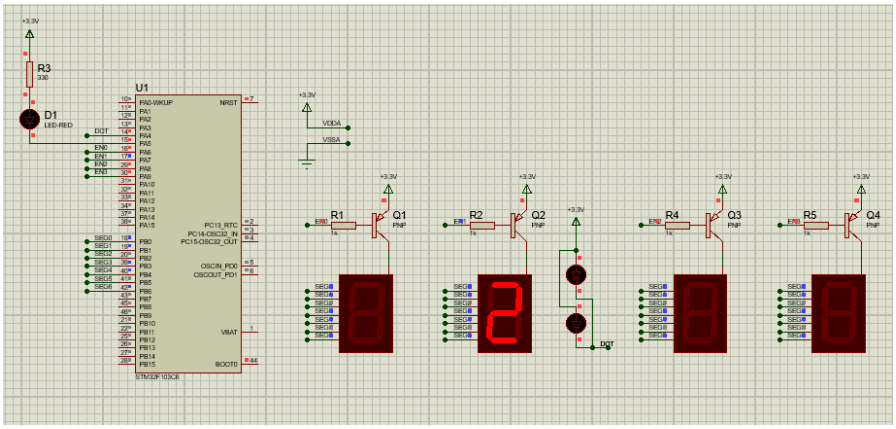
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2) {
        // Nhay DOT moi 1 giay
        dot_counter--;
        if (dot_counter <= 0) {
            dot_counter = 100;
            HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
            HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
        }

        // Chuyen LED 7 doan moi 500ms
        seg_counter--;
        if (seg_counter <= 0) {
            seg_counter = 50;
            switch (led7seg_state) {
                case 0:
                    HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, GPIO_PIN_SET);
                    HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, GPIO_PIN_SET);
                    HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, GPIO_PIN_SET);
                    display7SEG(1);
                    HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, GPIO_PIN_RESET); // Mo Q1
                    led7seg_state = 1;
```

```
28                      break;
29                  case 1:
30                      HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, GPIO_PIN_SET);
31                      HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, GPIO_PIN_SET);
32                      HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, GPIO_PIN_SET);
33                      display7SEG(2);
34                      HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, GPIO_PIN_RESET); // Mo Q2
35                      led7seg_state = 2;
36                      break;
37                  case 2:
38                      HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, GPIO_PIN_SET);
39                      HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, GPIO_PIN_SET);
40                      HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, GPIO_PIN_SET);
41                      display7SEG(3);
42                      HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, GPIO_PIN_RESET); // Mo Q3
43                      led7seg_state = 3;
44                      break;
45                  case 3:
46                      HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, GPIO_PIN_SET);
47                      HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, GPIO_PIN_SET);
48                      HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, GPIO_PIN_SET);
49                      display7SEG(0);
50                      HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, GPIO_PIN_RESET); // Mo Q4
51                      led7seg_state = 0; // Quay ve LED dau
52                      break;
53                  default:
54                      led7seg_state = 0;
55                      break;
56              }
57          }
58      }
59  }
60  /* USER CODE END 4 */
```

Mã nguồn ngắt timer cho bài tập 2

Kết quả mô phỏng:



Hình 4: Kết quả mô phỏng bài tập 2

Câu hỏi ngắn: *Tần số quét LED là bao nhiêu?*

Mỗi LED sáng trong $T_S = 0.5$s và có 4 LED, nên chu kỳ quét toàn bộ là $4 \times 0.5 = 2$s. Tần số quét là:

$$f = \frac{1}{4 \cdot T_S} = \frac{1}{4 \times 0.5} = 0.5\,\text{Hz}$$

4 Bài tập 3

Hiện thực hàm **update7SEG(int index)** để hiển thị giá trị trong mảng `led_buffer` lên LED 7 đoạn thứ `index`. Hàm này được gọi trong ngắt timer dưới dạng `update7SEG(index_led++)`. Biến `index_led` luôn được giữ trong khoảng 0–3.

Report 1: Hàm **update7SEG(int index)** hiển thị giá trị `led_buffer[index]` lên LED 7 đoạn thứ `index`. Trong mỗi trường hợp, ba LED còn lại được tắt trước (đưa chân EN lên mức 1 để ngắt transistor PNP), sau đó dữ liệu được xuất ra PB0–PB6 bằng hàm **display7SEG**, cuối cùng mở transistor của LED cần hiển thị.

```
/* USER CODE BEGIN PV */
const int MAX_LED = 4;
int index_led = 0;
int led_buffer[4] = {1, 2, 3, 4}; // buffer hien thi
/* USER CODE END PV */

/* USER CODE BEGIN 0 */
void update7SEG(int index) {
    switch (index) {
        case 0:
            // Tat cac LED 1, 2, 3 (PNP ngat o muc 1)
            HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, GPIO_PIN_SET);
            HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, GPIO_PIN_SET);
            HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, GPIO_PIN_SET);
            // Xuat du lieu cua led_buffer[0] ra SEG0..SEG6
            display7SEG(led_buffer[0]);
            // Mo LED 0 (PNP dan o muc 0)
            HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, GPIO_PIN_RESET);
            break;
        case 1:
            HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, GPIO_PIN_SET);
            HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, GPIO_PIN_SET);
            HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, GPIO_PIN_SET);
            display7SEG(led_buffer[1]);
            HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, GPIO_PIN_RESET);
            break;
        case 2:
            HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, GPIO_PIN_SET);
            HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, GPIO_PIN_SET);
            HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, GPIO_PIN_SET);
            display7SEG(led_buffer[2]);
            HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, GPIO_PIN_RESET);
            break;
        case 3:
            HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, GPIO_PIN_SET);
            HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, GPIO_PIN_SET);
            HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, GPIO_PIN_SET);
            display7SEG(led_buffer[3]);
            HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, GPIO_PIN_RESET);
            break;
        default:
```

```
            break;
    }
}
/* USER CODE END 0 */
```

Hàm **update7SEG(int index)**

Report 2: Trong hàm ngắt, sau mỗi 500ms (50 lần ngắt), hàm **update7SEG** được gọi với chỉ số `index_led` hiện tại, sau đó `index_led` được tăng lên và đưa về 0 khi vượt quá 3. Bộ đếm DOT từ bài tập 2 được giữ nguyên.

```
/* USER CODE BEGIN 4 */
int seg_counter = 50;      // 50 x 10ms = 500ms: chuyen LED 7 doan
int dot_counter = 100;     // 100 x 10ms = 1s: nhay DOT

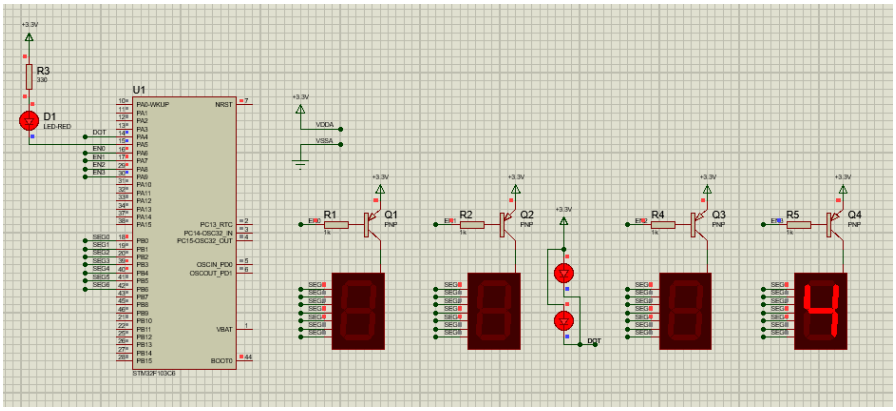
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2) {
        // Nhay DOT moi 1 giay
        dot_counter--;
        if (dot_counter <= 0) {
            dot_counter = 100;
            HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
            HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
        }

        // Chuyen LED 7 doan moi 500ms
        seg_counter--;
        if (seg_counter <= 0) {
            seg_counter = 50;
            update7SEG(index_led++);
            // Giu index_led trong khoang 0..3
            if (index_led >= MAX_LED) {
                index_led = 0;
            }
        }
    }
}
/* USER CODE END 4 */
```

Mã nguồn ngắt timer cho bài tập 3

Bài tập 3 và bài tập 4 được hiện thực chung trong một project. Mã nguồn trên GitHub đang được cấu hình cho bài tập 4 (`seg_counter = 25`); để chạy đúng bài tập 3, đổi hai giá trị 25 thành 50.

Kết quả mô phỏng với mảng `led_buffer` = {1, 2, 3, 4}:



Hình 5: Kết quả mô phỏng bài tập 3

5 Bài tập 4

Thay đổi chu kỳ gọi hàm **update7SEG** để tần số quét của 4 LED 7 đoạn là 1Hz, trong khi DOT vẫn tiếp tục nhấp nháy mỗi giây. Với tần số quét $f = 1\text{Hz}$ và 4 LED, thời gian sáng của mỗi LED là:

$$T_S = \frac{1}{4 \cdot f} = \frac{1}{4 \times 1} = 0.25\,\text{s} = 250\,\text{ms}$$

Do chu kỳ ngắt timer là 10ms, hàm **update7SEG** cần được gọi sau mỗi $\dfrac{250\,\text{ms}}{10\,\text{ms}} = 25$ lần ngắt.

Report 1: Mã nguồn trong hàm `HAL_TIM_PeriodElapsedCallback`. So với bài tập 3, chỉ giá trị nạp của bộ đếm `seg_counter` được đổi từ 50 thành 25. Bộ đếm `dot_counter` hoạt động độc lập nên DOT vẫn nhấp nháy với chu kỳ 1 giây.

```
/* USER CODE BEGIN 4 */
int seg_counter = 25;       // 25 x 10ms = 250ms: chuyen LED 7 doan (4 LED -> 1Hz)
int dot_counter = 100;      // 100 x 10ms = 1s: nhay DOT

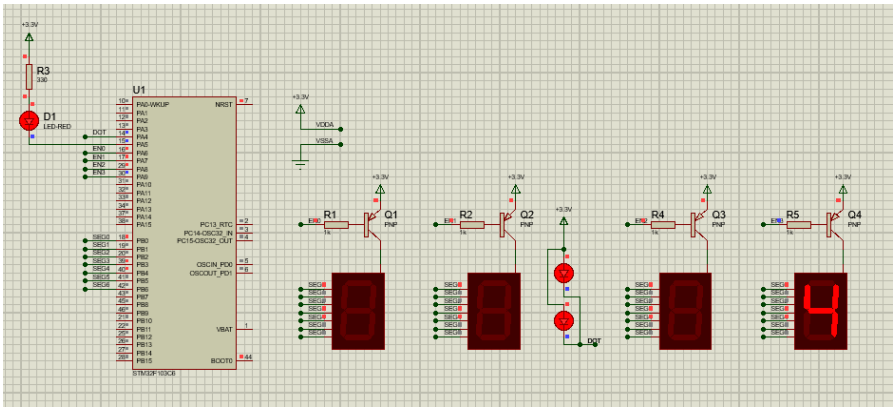
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2) {
        // Nhay DOT moi 1 giay
        dot_counter--;
        if (dot_counter <= 0) {
            dot_counter = 100;
            HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
            HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
        }

        // Chuyen LED 7 doan moi 250ms
        seg_counter--;
        if (seg_counter <= 0) {
            seg_counter = 25;
            update7SEG(index_led++);
            // Giu index_led trong khoang 0..3
            if (index_led >= MAX_LED) {
                index_led = 0;
            }
```

```
        }
    }
}
/* USER CODE END 4 */
```

Mã nguồn ngắt timer cho bài tập 4

Kết quả mô phỏng:



Hình 6: Kết quả mô phỏng bài tập 4

6 Bài tập 5

Hiện thực một đồng hồ số hiển thị **giờ** và **phút** trên 4 LED 7 đoạn: hai LED đầu hiển thị giờ, hai LED sau hiển thị phút. Thời điểm khởi tạo của đồng hồ là `15:08:50`. Bài tập này được phát triển tiếp từ bài tập 4, nên phần quét LED (250ms mỗi LED, tần số quét 1Hz) và nhấp nháy DOT trong hàm ngắt timer được giữ nguyên.

Hàm **updateClockBuffer** sinh giá trị cho mảng `led_buffer` từ hai biến `hour` và `minute`. Mỗi giá trị được tách thành chữ số hàng chục (phép chia `/ 10`) và chữ số hàng đơn vị (phép chia lấy dư `% 10`). Nhờ vậy, khi giá trị chỉ có một chữ số thì chữ số 0 được tự động thêm vào phía trước, ví dụ phút 8 được hiển thị là `08`.

Report 1: Mã nguồn của hàm **updateClockBuffer**:

```
/* USER CODE BEGIN PV */
// Thoi gian khoi tao cua dong ho
int hour = 15;
int minute = 8;
int second = 50;
/* USER CODE END PV */

/* USER CODE BEGIN 0 */
void updateClockBuffer(void) {
    // 2 LED dau hien thi Gio (HH)
    led_buffer[0] = hour / 10;
    led_buffer[1] = hour % 10;

```

```
    // 2 LED sau hien thi Phut (MM)
    led_buffer[2] = minute / 10;
    led_buffer[3] = minute % 10;
}
/* USER CODE END 0 */
```

Hàm **updateClockBuffer()**

Trong hàm main, giờ khởi tạo được nạp vào buffer trước khi vào vòng lặp. Sau đó, mỗi vòng lặp tăng biến `second`, cập nhật phút và giờ khi tràn, gọi **updateClockBuffer** và chờ 1 giây bằng `HAL_Delay(1000)`:

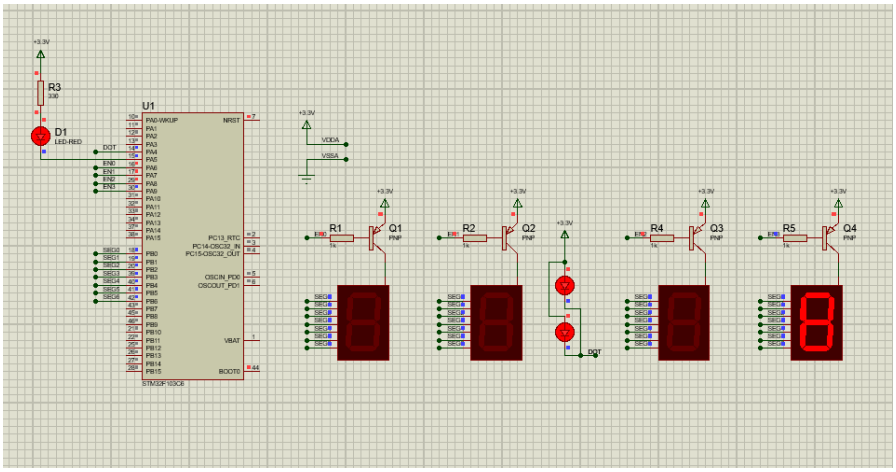
```
/* USER CODE BEGIN 2 */
// ... khoi dong timer va tat 4 transistor PNP nhu bai tap 2
// Nap gio khoi tao vao buffer truoc khi quet
updateClockBuffer();
/* USER CODE END 2 */

/* USER CODE BEGIN WHILE */
while (1)
{
    second++;
    if (second >= 60) {
        second = 0;
        minute++;
    }
    if (minute >= 60) {
        minute = 0;
        hour++;
    }
    if (hour >= 24) {
        hour = 0;
    }
    updateClockBuffer();
    HAL_Delay(1000);
    /* USER CODE END WHILE */
}
```

Mã nguồn hàm main cho bài tập 5

Mặc dù `HAL_Delay(1000)` làm vòng lặp chính bị chặn, việc quét LED vẫn diễn ra liên tục vì được thực hiện trong hàm ngắt timer.

Kết quả mô phỏng:



Hình 7: Kết quả mô phỏng đồng hồ số cho bài tập 5

7 Bài tập 6

Mục tiêu của bài tập này là giảm khối lượng xử lý trong hàm ngắt timer. Thời gian thực thi trong ngắt quá dài có thể gây ra ngắt lồng nhau và làm hệ thống hoạt động sai. Giải pháp là xây dựng một bộ định thời mềm (software timer): bộ đếm của nó được giảm sau mỗi lần ngắt (10ms), còn hàm main chỉ kiểm tra cờ báo hết thời gian. Cách này cho phép thay thế lệnh chặn `HAL_Delay` bằng cơ chế chờ không chặn (non-blocking).

Bài tập 6 và bài tập 7 được hiện thực chung trong một project. Software timer được tách thành module riêng gồm hai file `software_timer.h` và `software_timer.c`. Trong đó timer 0 phục vụ bài tập 6, timer 1 phục vụ bài tập 7. Hằng số `TIMER_CYCLE` được đặt bằng chu kỳ ngắt 10ms, nên tham số `duration` được tính theo đơn vị mili giây.

```h
#ifndef INC_SOFTWARE_TIMER_H_
#define INC_SOFTWARE_TIMER_H_

extern volatile int timer0_flag;
extern volatile int timer1_flag;

void setTimer0(int duration);
void setTimer1(int duration);
void timer_run(void);

#endif /* INC_SOFTWARE_TIMER_H_ */
```

software_timer.h

```c
#include "software_timer.h"

#define TIMER_CYCLE 10     // TIM2 ngat moi 10ms

// Timer 0: nhay LED_RED (bai tap 6)
volatile int timer0_counter = 0;
volatile int timer0_flag = 0;

```

```c
// Timer 1: dem giay cho dong ho va nhay DOT (bai tap 7)
volatile int timer1_counter = 0;
volatile int timer1_flag = 0;

void setTimer0(int duration){
    timer0_counter = duration / TIMER_CYCLE;
    timer0_flag = 0;
}

void setTimer1(int duration){
    timer1_counter = duration / TIMER_CYCLE;
    timer1_flag = 0;
}

// Goi trong ngat timer moi 10ms
void timer_run(){
    if(timer0_counter > 0){
        timer0_counter--;
        if(timer0_counter == 0) timer0_flag = 1;
    }
    if(timer1_counter > 0){
        timer1_counter--;
        if(timer1_counter == 0) timer1_flag = 1;
    }
}
```

software_timer.c

Hàm `timer_run()` được gọi trong hàm ngắt timer. Trong hàm main, timer 0 được khởi tạo bằng `setTimer0(1000)`, sau đó vòng lặp kiểm tra cờ `timer0_flag` để nháy LED tại PA5 với chu kỳ 2 giây:

```c
/* USER CODE BEGIN 2 */
setTimer0(1000);
/* USER CODE END 2 */

/* USER CODE BEGIN WHILE */
while (1){
    if(timer0_flag == 1){
        HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
        setTimer0(2000);
    }
}
/* USER CODE END WHILE */

/* USER CODE BEGIN 4 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){
    timer_run();
    // ... ma nguon quet LED 7 doan (bai tap 7)
}
/* USER CODE END 4 */
```

Sử dụng software timer để nháy LED

Report 1: *Nếu thiếu dòng 1 (`setTimer0(1000)`) thì điều gì xảy ra và tại sao?*

LED sẽ không bao giờ nhấp nháy. Khi khởi tạo, `timer0_counter = 0` và `timer0_flag = 0`. Trong `timer_run()`, điều kiện `timer0_counter > 0` luôn sai nên cờ `timer0_flag` không bao giờ được bật. Vì vậy khối lệnh trong `while` không bao giờ được thực thi, kể cả lệnh `setTimer0(2000)`.

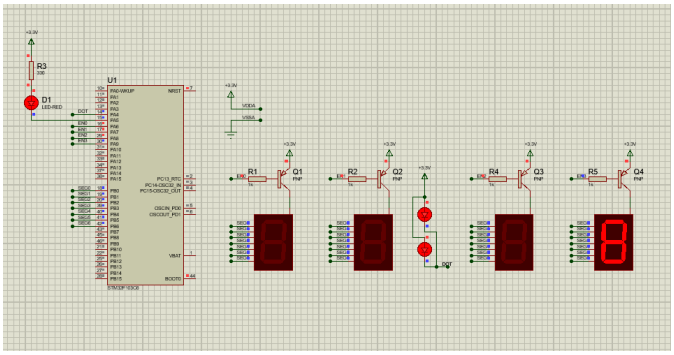
Report 2: *Nếu dòng 1 được đổi thành `setTimer0(1)` thì điều gì xảy ra và tại sao?*

Kết quả giống câu 1: LED không nhấp nháy. Do phép chia nguyên, `timer0_counter = 1 / 10 = 0`, nên trạng thái của software timer giống hệt trường hợp không gọi `setTimer0`. Bộ đếm bằng 0 nên cờ không bao giờ được bật.

Report 3: *Nếu dòng 1 được đổi thành `setTimer0(10)` thì điều gì thay đổi so với 2 câu trên và tại sao?*

Lần này `timer0_counter = 10 / 10 = 1 > 0`. Ở lần ngắt kế tiếp (sau tối đa 10ms), bộ đếm giảm về 0 và cờ `timer0_flag` được bật. LED đổi trạng thái gần như ngay khi khởi động, sau đó được nạp `setTimer0(2000)` và nhấp nháy đều đặn mỗi 2 giây. Khác với hai câu trên, hệ thống hoạt động bình thường vì giá trị truyền vào ít nhất bằng một chu kỳ ngắt (`TIMER_CYCLE`).

Kết quả mô phỏng:



Hình 8: Kết quả mô phỏng software timer cho bài tập 6

8 Bài tập 7

Nâng cấp đồng hồ số ở bài tập 5: việc cập nhật giờ, phút, giây được điều khiển bởi software timer 1 (chu kỳ 1 giây) và lệnh `HAL_Delay(1000)` được loại bỏ. Việc nhấp nháy DOT (PA4) cũng được chuyển từ hàm ngắt vào hàm main và dùng chung timer 1. Hàm ngắt timer lúc này chỉ còn cập nhật software timer và quét LED 7 đoạn.

Report 1: Mã nguồn trong vòng lặp while của hàm main:

```
/* USER CODE BEGIN 2 */
// ... khoi dong timer va tat 4 transistor PNP nhu bai tap 2
// Nap gio khoi tao vao buffer truoc khi quet
updateClockBuffer();

setTimer0(1000);   // Bai 6: LED_RED doi trang thai lan dau sau 1s
setTimer1(1000);   // Bai 7: dem giay va nhay DOT moi 1s
/* USER CODE END 2 */

/* USER CODE BEGIN WHILE */
while (1)
{
    // Bai 6: nhay LED_RED bang software timer 0
```

```
    if (timer0_flag == 1) {
        HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
        setTimer0(2000);
    }

    // Bai 7: cap nhat dong ho va nhay DOT bang software timer 1
    if (timer1_flag == 1) {
        setTimer1(1000);
        HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);

        second++;
        if (second >= 60) {
            second = 0;
            minute++;
        }
        if (minute >= 60) {
            minute = 0;
            hour++;
        }
        if (hour >= 24) {
            hour = 0;
        }
        updateClockBuffer();
    }
    /* USER CODE END WHILE */
}
```

Mã nguồn hàm main cho bài tập 7

Hàm ngắt timer sau khi chuyển DOT vào hàm main:

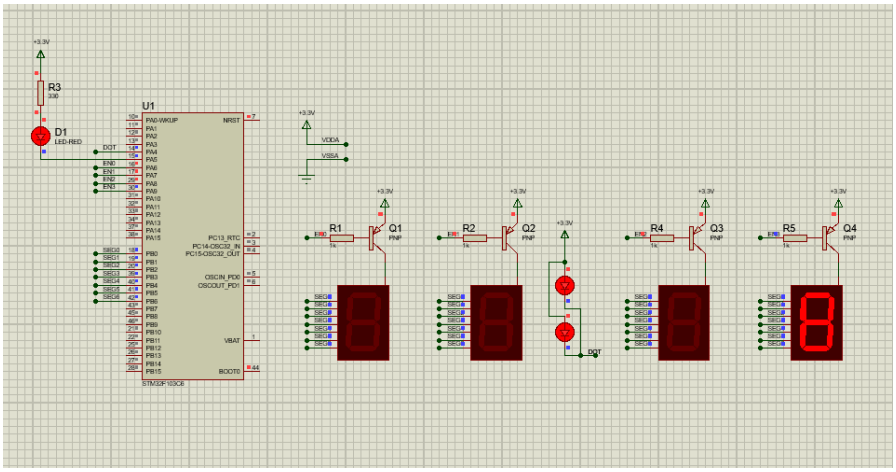
```
/* USER CODE BEGIN 4 */
int seg_counter = 25;      // 25 x 10ms = 250ms: chuyen LED 7 doan (4 LED -> 1Hz)

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2) {
        // Cap nhat cac software timer
        timer_run();

        // Chuyen LED 7 doan moi 250ms
        seg_counter--;
        if (seg_counter <= 0) {
            seg_counter = 25;
            update7SEG(index_led++);
            // Giu index_led trong khoang 0..3
            if (index_led >= MAX_LED) {
                index_led = 0;
            }
        }
    }
}
/* USER CODE END 4 */
```

Mã nguồn ngắt timer cho bài tập 7

Kết quả mô phỏng:



Hình 9: Kết quả mô phỏng bài tập 7

9 Bài tập 8

Bài tập này chuyển nốt hàm **update7SEG()** từ hàm ngắt timer vào hàm main. Sau khi chuyển, hàm ngắt timer chỉ còn nhiệm vụ cập nhật các software timer thông qua `timer_run()`. Mọi xử lý khác (quét LED, cập nhật đồng hồ, nhấp nháy LED) đều được thực hiện trong vòng lặp vô tận của hàm main, nên thời gian thực thi trong ngắt là ngắn nhất có thể. Bài tập này được phát triển tiếp từ bài tập 7, với một software timer mới (timer 2) được thêm vào. Hệ thống sử dụng ba software timer:

- **Timer 0** (2000ms): nhấp nháy LED_RED (từ bài tập 6).
- **Timer 1** (1000ms): tăng giây, cập nhật đồng hồ và nhấp nháy DOT (từ bài tập 7).
- **Timer 2** (250ms): quét 4 LED 7 đoạn, tần số quét 1Hz.

Report 1: Mã nguồn trong hàm main:

```
/* USER CODE BEGIN 2 */
HAL_TIM_Base_Start_IT(&htim2);

// Khoa tat ca cac transistor PNP luc khoi dong (SET la tat nguon LED)
HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, GPIO_PIN_SET);
HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, GPIO_PIN_SET);
HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, GPIO_PIN_SET);
HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, GPIO_PIN_SET);

// Nap gio khoi tao vao buffer truoc khi quet
updateClockBuffer();

setTimer0(1000);    // Bai 6: LED_RED doi trang thai lan dau sau 1s
setTimer1(1000);    // Bai 7: dem giay va nhay DOT moi 1s
setTimer2(250);     // Bai 8: quet LED 7 doan moi 250ms (4 LED -> 1Hz)
/* USER CODE END 2 */

```

```c
/* USER CODE BEGIN WHILE */
while (1)
{
    // Bai 6: nhay LED_RED bang software timer 0
    if (timer0_flag == 1) {
        HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
        setTimer0(2000);
    }

    // Bai 7: cap nhat dong ho va nhay DOT bang software timer 1
    if (timer1_flag == 1) {
        setTimer1(1000);
        HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);

        second++;
        if (second >= 60) {
            second = 0;
            minute++;
        }
        if (minute >= 60) {
            minute = 0;
            hour++;
        }
        if (hour >= 24) {
            hour = 0;
        }
        updateClockBuffer();
    }

    // Bai 8: quet LED 7 doan bang software timer 2
    if (timer2_flag == 1) {
        setTimer2(250);
        update7SEG(index_led++);
        // Giu index_led trong khoang 0..3
        if (index_led >= MAX_LED) {
            index_led = 0;
        }
    }
    /* USER CODE END WHILE */
}
```

Mã nguồn hàm main cho bài tập 8

Hàm ngắt timer sau khi chuyển toàn bộ xử lý vào hàm main:

```c
/* USER CODE BEGIN 4 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2) {
        // Cap nhat cac software timer
        timer_run();
    }
}
/* USER CODE END 4 */
```

Mã nguồn ngắt timer cho bài tập 8

Do sử dụng thêm software timer thứ ba, module `software_timer` được mở rộng như sau:

```c
#ifndef INC_SOFTWARE_TIMER_H_
#define INC_SOFTWARE_TIMER_H_
```

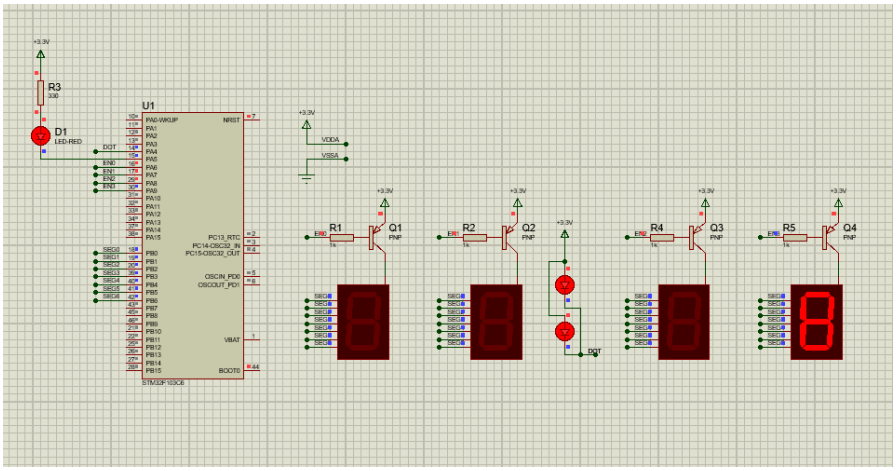
```
3
4  extern volatile int timer0_flag;
5  extern volatile int timer1_flag;
6  extern volatile int timer2_flag;
7
8  void setTimer0(int duration);
9  void setTimer1(int duration);
10 void setTimer2(int duration);
11 void timer_run(void);
12
13 #endif /* INC_SOFTWARE_TIMER_H_ */
```

software_timer.h

```
1  #include "software_timer.h"
2
3  #define TIMER_CYCLE 10     // TIM2 ngat moi 10ms
4
5  // Timer 0: nhay LED_RED (bai tap 6)
6  volatile int timer0_counter = 0;
7  volatile int timer0_flag = 0;
8
9  // Timer 1: dem giay cho dong ho va nhay DOT (bai tap 7)
10 volatile int timer1_counter = 0;
11 volatile int timer1_flag = 0;
12
13 // Timer 2: quet LED 7 doan (bai tap 8)
14 volatile int timer2_counter = 0;
15 volatile int timer2_flag = 0;
16
17 void setTimer0(int duration){
18     timer0_counter = duration / TIMER_CYCLE;
19     timer0_flag = 0;
20 }
21
22 void setTimer1(int duration){
23     timer1_counter = duration / TIMER_CYCLE;
24     timer1_flag = 0;
25 }
26
27 void setTimer2(int duration){
28     timer2_counter = duration / TIMER_CYCLE;
29     timer2_flag = 0;
30 }
31
32 // Goi trong ngat timer moi 10ms
33 void timer_run(){
34     if(timer0_counter > 0){
35         timer0_counter--;
36         if(timer0_counter == 0) timer0_flag = 1;
37     }
38     if(timer1_counter > 0){
39         timer1_counter--;
40         if(timer1_counter == 0) timer1_flag = 1;
41     }
42     if(timer2_counter > 0){
43         timer2_counter--;
44         if(timer2_counter == 0) timer2_flag = 1;
45     }
46 }
```

software_timer.c

Kết quả mô phỏng:



Hình 10: Kết quả mô phỏng bài tập 8

10 Bài tập 9

Bài tập này bổ sung một ma trận LED 8x8 (**MATRIX-8X8-RED**) vào hệ thống, cùng với vi mạch **ULN2803**. ULN2803 là mảng transistor NPN, dùng để dẫn dòng cho từng cột của ma trận. Tín hiệu chọn cột ENM0–ENM7 được nối vào các chân PA2, PA3, PA10–PA15. Tín hiệu dữ liệu hàng ROW0–ROW7 được nối từ PB8 đến PB15. Tại mỗi thời điểm chỉ một cột được bật; ma trận được quét lần lượt từng cột, tương tự như 4 LED 7 đoạn.

Bài tập 9 và bài tập 10 được hiện thực chung trong một project, phát triển tiếp từ bài tập 8. Với chu kỳ ngắt 10ms như các bài trước, mỗi cột sáng 10ms và toàn bộ ma trận chỉ được quét với tần số $\frac{1}{8\times 10\,\text{ms}} = 12.5\text{Hz}$, khiến hình ảnh bị nhấp nháy rõ rệt. Ngược lại, nếu quét quá nhanh thì phần mềm Proteus không mô phỏng kịp theo thời gian thực. Để cân bằng hai yếu tố này, Timer 2 được cấu hình lại với `Counter Period = 4`:

$$f_{\text{ngắt}} = \frac{8\,\text{MHz}}{(7999+1)(4+1)} = 200\,\text{Hz} \;\Rightarrow\; T_{\text{ngắt}} = 5\,\text{ms}$$

Hằng số `TIMER_CYCLE` trong module software timer được đổi thành 5. Vì tham số của các hàm `setTimerX` được tính theo mili giây, các software timer của bài tập 6–8 vẫn giữ nguyên chu kỳ. Ma trận được quét 5ms mỗi cột, nên tần số quét toàn bộ là $\frac{1}{8\times 5\,\text{ms}} = 25\text{Hz}$. Đây là mức Proteus mô phỏng kịp mà ký tự vẫn hiển thị rõ ràng.

Module software timer được mở rộng thêm timer 3 (quét ma trận) và timer 4 (hiệu ứng ở bài tập 10). Các hàm `setTimer0` – `setTimer2` giữ nguyên như bài tập 8:

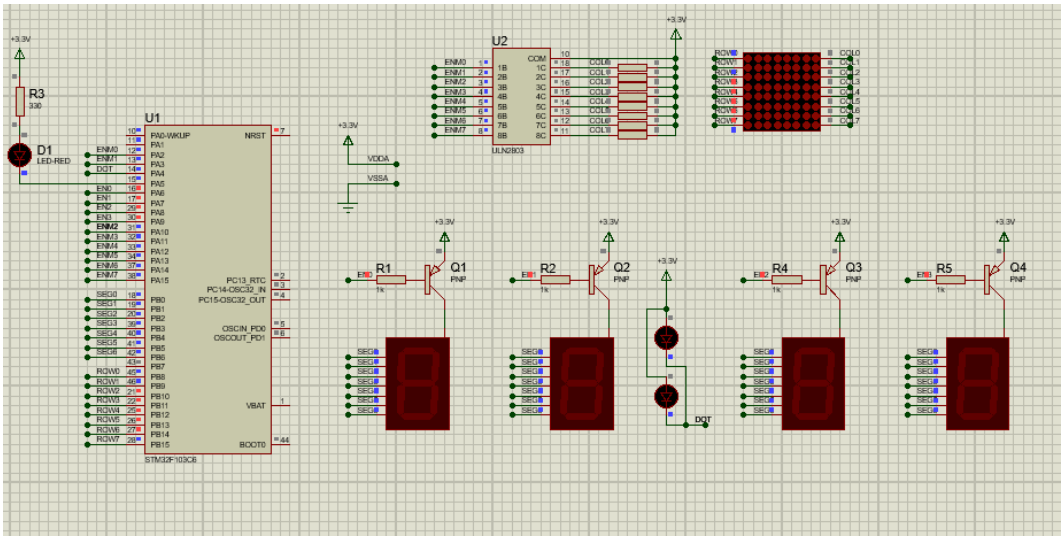
```
#define TIMER_CYCLE 5      // TIM2 ngat moi 5ms

// Timer 3: quet ma tran LED (bai tap 9)
volatile int timer3_counter = 0;
volatile int timer3_flag = 0;
```

```
6
7  // Timer 4: buoc hieu ung chu chay (bai tap 10)
8  volatile int timer4_counter = 0;
9  volatile int timer4_flag = 0;
10
11 void setTimer3(int duration){
12     timer3_counter = duration / TIMER_CYCLE;
13     timer3_flag = 0;
14 }
15
16 void setTimer4(int duration){
17     timer4_counter = duration / TIMER_CYCLE;
18     timer4_flag = 0;
19 }
20
21 // Goi trong ngat timer moi TIMER_CYCLE ms
22 void timer_run(){
23     // ... timer 0, 1, 2 nhu bai tap 8
24     if(timer3_counter > 0){
25         timer3_counter--;
26         if(timer3_counter == 0) timer3_flag = 1;
27     }
28     if(timer4_counter > 0){
29         timer4_counter--;
30         if(timer4_counter == 0) timer4_flag = 1;
31     }
32 }
```

Phần mở rộng của software_timer.c

Report 1: Sơ đồ mô phỏng trên Proteus:



Hình 11: Sơ đồ nguyên lý hệ thống có ma trận LED cho bài tập 9

Link github cho file mô phỏng bài tập 9 (dùng chung cho bài tập 10): https://github.com/qanetta/Microprocessor-Lab/blob/main/lab2/proteus_files/Ex9-10_led-matrix.pdsprj

Report 2: Hàm **updateLEDMatrix(int index)**. Mỗi phần tử `matrix_buffer[i]` chứa dữ liệu của **cột** thứ i; bit thứ k tương ứng với hàng ROWk. Hàm phụ **setRowValues** ghi byte dữ liệu lên 8 bit cao của thanh ghi `GPIOB->ODR` (PB8–PB15) và giữ nguyên 8 bit thấp, vốn đang dùng cho LED 7 đoạn. Trong **updateLEDMatrix**, toàn bộ 8 cột được tắt trước để tránh bóng mờ, sau đó dữ liệu được xuất ra các hàng và chỉ một cột tương ứng được bật.

Để hiển thị chữ **"A"**, ảnh chữ được mã hóa theo cột như sau (ROW0 ở trên cùng):

Cột	0	1	2	3	4	5	6	7
Giá trị	0x00	0x7C	0x0A	0x09	0x09	0x0A	0x7C	0x00

```c
/* USER CODE BEGIN PV */
const int MAX_LED_MATRIX = 8;
int index_led_matrix = 0;

// Chu A ma hoa theo COT: phan tu i la cot i, bit k la ROWk (ROW0 o tren)
const uint8_t CHAR_A[8] = {0x00, 0x7C, 0x0A, 0x09, 0x09, 0x0A, 0x7C, 0x00};
uint8_t matrix_buffer[8] = {0x00, 0x7C, 0x0A, 0x09, 0x09, 0x0A, 0x7C, 0x00};
/* USER CODE END PV */

/* USER CODE BEGIN 0 */
void setRowValues(uint8_t data) {
    // ghi vao ODR (PB8..PB15), giu nguyen PB0..PB7
    GPIOB->ODR = (GPIOB->ODR & 0x00FF) | ((uint16_t)data << 8);
}

void updateLEDMatrix(int index) {
    // 1. Tat toan bo 8 cot truoc (ENM0 -> ENM7)
    HAL_GPIO_WritePin(GPIOA, ENM0_Pin|ENM1_Pin|ENM2_Pin|ENM3_Pin|
                             ENM4_Pin|ENM5_Pin|ENM6_Pin|ENM7_Pin, GPIO_PIN_RESET);

    // 2. Xuat du lieu cua cot tuong ung ra cac hang PB8..PB15
    setRowValues(matrix_buffer[index]);

    // 3. Kich mo DUY NHAT mot cot theo index
    switch (index) {
        case 0: HAL_GPIO_WritePin(GPIOA, ENM0_Pin, GPIO_PIN_SET); break;
        case 1: HAL_GPIO_WritePin(GPIOA, ENM1_Pin, GPIO_PIN_SET); break;
        case 2: HAL_GPIO_WritePin(GPIOA, ENM2_Pin, GPIO_PIN_SET); break;
        case 3: HAL_GPIO_WritePin(GPIOA, ENM3_Pin, GPIO_PIN_SET); break;
        case 4: HAL_GPIO_WritePin(GPIOA, ENM4_Pin, GPIO_PIN_SET); break;
        case 5: HAL_GPIO_WritePin(GPIOA, ENM5_Pin, GPIO_PIN_SET); break;
        case 6: HAL_GPIO_WritePin(GPIOA, ENM6_Pin, GPIO_PIN_SET); break;
        case 7: HAL_GPIO_WritePin(GPIOA, ENM7_Pin, GPIO_PIN_SET); break;
        default: break;
    }
}
/* USER CODE END 0 */
```

Hàm **updateLEDMatrix(int index)**

Hàm **updateLEDMatrix** được gọi trong hàm main bằng software timer 3, mỗi 5ms quét một cột:

```c
/* USER CODE BEGIN 2 */
// ... phan khoi tao cua bai tap 8
// Tat toan bo 8 cot LED Matrix ban dau
```

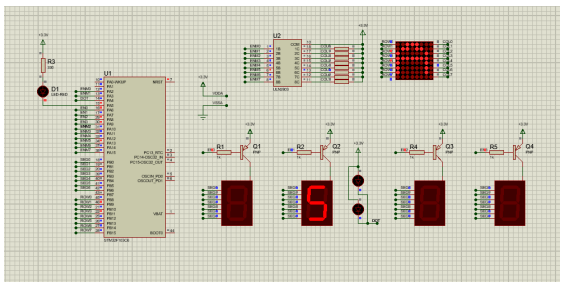
```
HAL_GPIO_WritePin(GPIOA, ENM0_Pin | ENM1_Pin | ENM2_Pin | ENM3_Pin |
                         ENM4_Pin | ENM5_Pin | ENM6_Pin | ENM7_Pin, GPIO_PIN_RESET);
setTimer3(5);      // Bai 9: quet ma tran 5ms/cot (8 cot -> 25Hz)
/* USER CODE END 2 */

/* USER CODE BEGIN WHILE */
while (1)
{
    // ... cac khoi xu ly cua bai tap 6, 7, 8

    // Bai 9: quet ma tran LED bang software timer 3
    if (timer3_flag == 1) {
        setTimer3(5);
        updateLEDMatrix(index_led_matrix++);
        if (index_led_matrix >= MAX_LED_MATRIX) {
            index_led_matrix = 0;
        }
    }
    /* USER CODE END WHILE */
}
```

Gọi hàm quét ma trận LED trong main

Kết quả hiển thị chữ "A"trên ma trận LED (`ENABLE_ANIMATION = 0`):



Hình 12: Kết quả hiển thị chữ A trên ma trận LED 8x8

11 Bài tập 10

Tạo hiệu ứng chuyển động (animation) cho ký tự trên ma trận LED: chữ **"A"** chạy từ phải sang trái (hiệu ứng chạy chữ – marquee).

Report 1: *Mô tả giải pháp.* Ý tưởng là coi chữ A như một dải mẫu dài 16 cột (`ANIM_LEN`), gồm 8 cột ảnh chữ A (mảng hằng `CHAR_A`) nối tiếp 8 cột trống. Ma trận LED đóng vai trò một "cửa sổ"rộng 8 cột nhìn vào dải mẫu này, bắt đầu tại vị trí `anim_offset`. Mỗi lần gọi, hàm **updateAnimation()** thực hiện hai việc:

1. Nạp lại toàn bộ `matrix_buffer`: cột i của ma trận lấy dữ liệu tại vị trí (`anim_offset` + i) mod 16 của dải mẫu. Vị trí nhỏ hơn 8 lấy từ `CHAR_A`, ngược lại là cột trống `0x00`.
2. Tăng `anim_offset` thêm 1 (theo modulo 16), làm cửa sổ trượt sang phải một cột. Kết quả là hình ảnh trên ma trận dịch sang trái một cột.

Vì vậy chữ A đi vào từ mép phải, trôi qua ma trận, ra khỏi mép trái, rồi lặp lại sau 16 bước. Hàm được gọi bởi software timer 4 mỗi 200ms, nên một vòng chạy chữ mất $16 \times 200 = 3.2$ giây. Mỗi vị trí của chữ được giữ trong 200ms, tương ứng 5 lần quét toàn bộ ma trận (40ms/lần), nên chữ hiển thị rõ khi đang chuyển động. Việc quét ma trận (timer 3, 5ms/cột) và việc cập nhật hiệu ứng (timer 4) độc lập với nhau: tốc độ chạy chữ không ảnh hưởng đến tần số quét. Nhờ dùng mảng hằng `CHAR_A` làm mẫu gốc, dữ liệu chữ không bị biến đổi tích lũy sau nhiều vòng lặp.

```
/* USER CODE BEGIN PD */
#define ENABLE_ANIMATION 1   // 0: bai 9 (chu A dung yen), 1: bai 10 (chu chay)
/* USER CODE END PD */

/* USER CODE BEGIN PV */
// Hieu ung chu chay: 8 cot chu A + 8 cot trong
#define ANIM_LEN 16
int anim_offset = 0;
/* USER CODE END PV */

/* USER CODE BEGIN 0 */
void updateAnimation(void)
{
    // Dich ca ma tran sang trai 1 cot: cot i lay mau tai vi tri (offset + i)
    for (int i = 0; i < MAX_LED_MATRIX; i++) {
        int idx = (anim_offset + i) % ANIM_LEN;
        // 8 vi tri dau la chu A, 8 vi tri sau la cot trong
        matrix_buffer[i] = (idx < 8) ? CHAR_A[idx] : 0x00;
    }
    anim_offset = (anim_offset + 1) % ANIM_LEN;
}
/* USER CODE END 0 */
```

Hàm **updateAnimation()**

Trong hàm main, hiệu ứng được bật/tắt bằng hằng `ENABLE_ANIMATION` và được điều khiển bởi software timer 4:

```
/* USER CODE BEGIN 2 */
// ... phan khoi tao cua bai tap 8 va 9
setTimer4(200);    // Bai 10: dich chu moi 200ms
/* USER CODE END 2 */

/* USER CODE BEGIN WHILE */
while (1)
{
    // ... cac khoi xu ly cua bai tap 6, 7, 8, 9

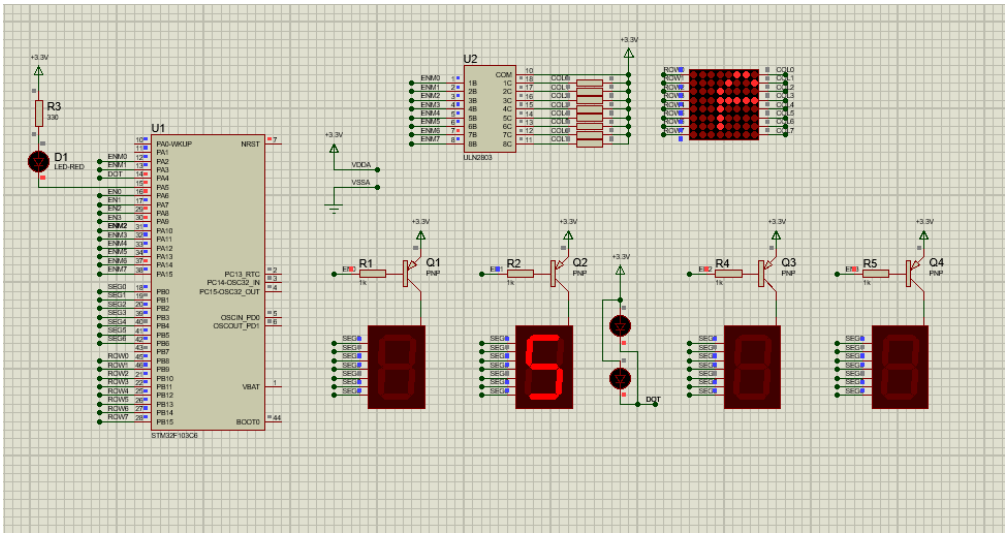
#if ENABLE_ANIMATION
    // Bai 10: hieu ung chu chay bang software timer 4
    if (timer4_flag == 1) {
        setTimer4(200);
        updateAnimation();
    }
#endif
    /* USER CODE END WHILE */
}

/* USER CODE BEGIN 4 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2) {
        // Cap nhat cac software timer
```

```
25          timer_run();
26      }
27  }
28  /* USER CODE END 4 */
```

Mã nguồn hàm main cho bài tập 10

Kết quả mô phỏng hiệu ứng chạy chữ:



Hình 13: Hiệu ứng chạy chữ A trên ma trận LED (chạy về mép phải, sau đó chạy từ trái sang)

12 Mã nguồn cho bài thực hành 2

Toàn bộ mã nguồn (STM32CubeIDE project) cho các bài tập ở bài thực hành 2: https://github.com/qanetta/Microprocessor-Lab/tree/main/lab2/stm32cube_src

Các bài tập có nội dung nối tiếp nhau được hiện thực chung trong một project, cụ thể như sau:

- `Ex1`: bài tập 1.
- `Ex2`: bài tập 2.
- `Ex3-4`: bài tập 3 và 4 (đang cấu hình cho bài tập 4).
- `Ex5`: bài tập 5.
- `Ex6-7`: bài tập 6 và 7.
- `Ex8`: bài tập 8.
- `Ex9-10`: bài tập 9 và 10 (chuyển đổi bằng hằng `ENABLE_ANIMATION`).

Toàn bộ file mô phỏng Proteus cho bài thực hành 2: https://github.com/qanetta/Microprocessor-Lab/tree/main/lab2/proteus_files